■■ BIS AI Assistant

Bureau of Indian Standards — Intelligent Conversational Platform

Full Project Technical Documentation
User Flow · Technology Stack · Architecture · Impact Analysis

■ Next.js 14	■ Spring Boot 3	■ FastAPI RAG	■■ Qdrant Vector DB	■ PostgreSQL 16

Smart India Hackathon 2026 Project

Prepared: September 2026

■ Table of Contents

1. Executive Summary 3
2. Project Overview & Purpose 3
3. System Architecture Diagram 4
4. User Flow (End-to-End Walkthrough) 5
5. RAG Pipeline — AI Query Processing 6
6. Technology Stack — Deep Dive 7
6.1 Frontend Layer 7
6.2 Backend Layer 8
6.3 AI / RAG Layer 9
6.4 Database Layer 10
6.5 Infrastructure & Deployment 11
7. Technology Comparison Table 12
8. Impact Analysis 13
9. Data Flow Summary 14
10. Security Architecture 14

1. Executive Summary

The **BIS AI Assistant** is an AI-powered, multilingual conversational platform built for the **Bureau of Indian Standards (BIS)** — India's national standards body. It enables manufacturers, MSMEs, consumers, and students to instantly query Indian Standards (IS documents), understand BIS certification schemes, and navigate hallmarking and licensing procedures — all through a natural language chat interface.

The system uses **Retrieval-Augmented Generation (RAG)** to answer questions exclusively from official BIS documents, ensuring factual accuracy with clause-level citations. It supports both English and Hindi, making it accessible to a wider population.

Metric	Value
Target Users	Manufacturers, MSMEs, Consumers, Students
Languages Supported	English + Hindi (auto-detect & translate)
AI Architecture	Retrieval-Augmented Generation (RAG)
Vector Search	Qdrant with dense embeddings (1024-dim)
LLM Backend	OpenAI GPT / Groq (configurable)
Deployment	6-service Docker Compose stack
Authentication	JWT Stateless (Spring Security)

2. Project Overview & Purpose

BIS maintains thousands of Indian Standards (IS documents) covering safety, quality, and performance requirements for products sold in India. Navigating these documents manually is time-consuming and technically demanding. The BIS AI Assistant solves this by:

- ■ **Semantic Search** — finding the right IS document even with approximate or Hindi queries
- ■ **Clause-level Citations** — every answer references the exact IS number and clause
- ■ **Multilingual** — queries in Hindi are auto-detected, translated, and answered back in Hindi
- ■ **Conversational Memory** — follow-up questions retain context of previous exchanges
- ■ **Document Ingestion** — admins can upload PDF standards and BIS crawls bis.gov.in automatically
- ■ **Secure Authentication** — JWT-based login with role-based admin access
- ■ **Streaming Responses** — answers stream token-by-token via Server-Sent Events (SSE)

3. System Architecture Diagram

The system is composed of **6 Docker services** organized into two networks — bis-internal (private, service-to-service) and bis-external (public-facing via NGINX). Each service has health checks and restarts automatically on failure.

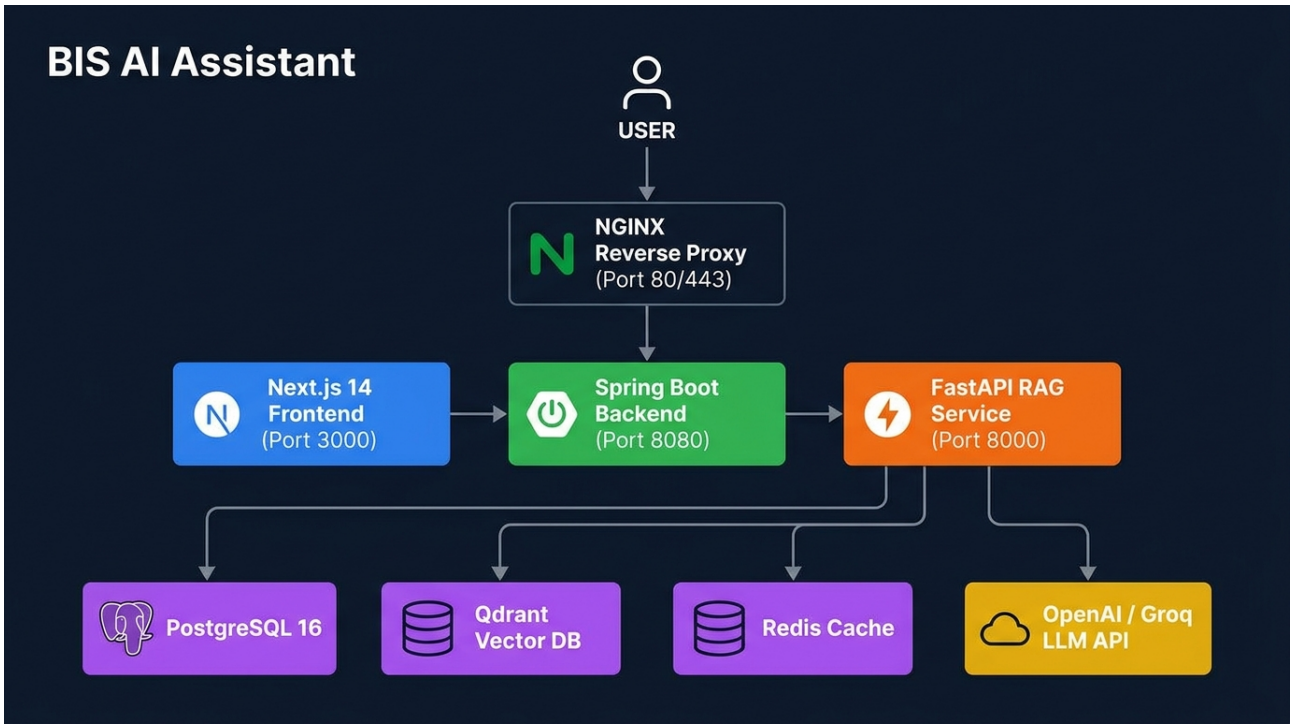
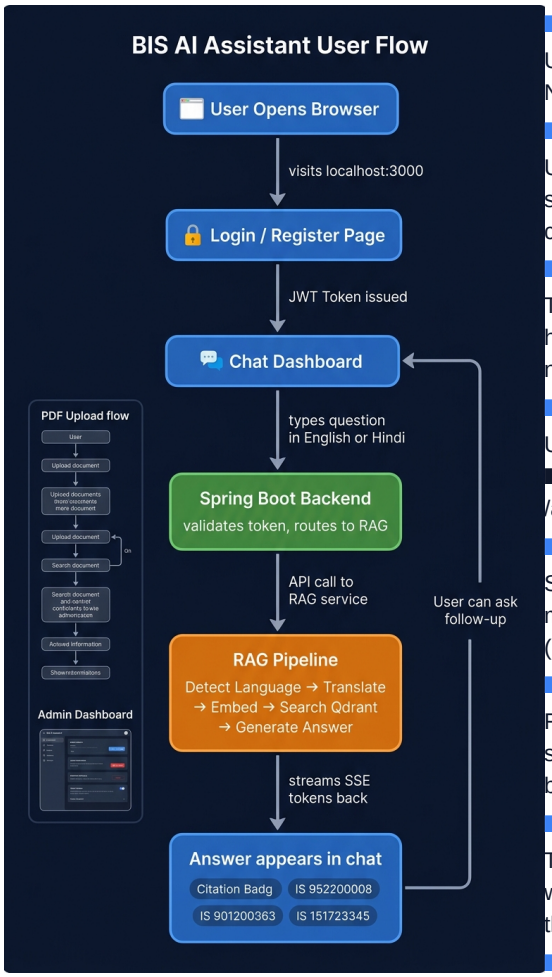


Figure 1 — BIS AI Assistant System Architecture

Service	Technology	Port	Role
nginx	NGINX 1.27	80 / 443	Reverse proxy, SSL termination, routing
frontend	Next.js 14	3000	React UI — chat, auth, admin dashboard
backend	Spring Boot 3 + Java 21	8080	REST API, auth, JWT, session management
rag	FastAPI + LangChain	8000	AI pipeline — embed, search, generate
qdrant	Qdrant v1.12.1	6333	Vector database for semantic search
postgres	PostgreSQL 16	5432	User data, sessions, document metadata
redis	Redis 7	6379	Conversation memory cache, response caching

4. User Flow — End-to-End Walkthrough

Below is the complete journey of a user interacting with the BIS AI Assistant — from opening the browser to receiving a cited, AI-generated answer.



■ Step 1 — Open App

User navigates to http://localhost:3000 (or the production URL). NGINX routes the request to the Next.js frontend container.

■ Step 2 — Authentication

User registers or logs in. Spring Boot validates credentials, issues a signed JWT token (RS256), and stores it in a secure HTTP-only cookie via js-cookie.

■ Step 3 — Chat Dashboard

The main chat interface loads. The user sees their conversation history (fetched via SWR from the backend REST API) and can start a new query.

■ Step 4 — Ask a Question

User types a question in English or Hindi, e.g. 'ISI mark ■■ ■■■ ■■■■■ ■■■■■ ■■■■■?' ChatInput.tsx sends a POST to /api/v1/chat/stream.

■ Step 5 — Backend Routes Request

Spring Boot verifies the JWT, validates the request body, saves the message to PostgreSQL, and forwards the request to the RAG service (FastAPI at port 8000).

■ Step 6 — RAG Pipeline Processes

FastAPI detects Hindi → translates → embeds the English query → searches Qdrant → builds context → calls the LLM → streams tokens back via SSE.

■ Step 7 — Streaming Answer

The frontend receives SSE token events and renders them in real time with react-markdown. Citation badges (IS numbers) appear first before the answer text.

■ Step 8 — Follow-Up Questions

Redis stores the last N conversation turns. The next query includes history so the LLM can answer follow-up questions with full context.

■ Admin Flow — Document Ingestion

Admins upload PDFs via the admin dashboard. The backend sends the file to /rag/ingest/pdf. PyPDF + pdfplumber extract text → chunked → embedded → stored in Qdrant.

Figure 2 — User Journey Flow Diagram

5. RAG Pipeline — AI Query Processing

The RAG (Retrieval-Augmented Generation) pipeline is the core AI component. It ensures answers are always grounded in official BIS documents rather than model hallucinations.

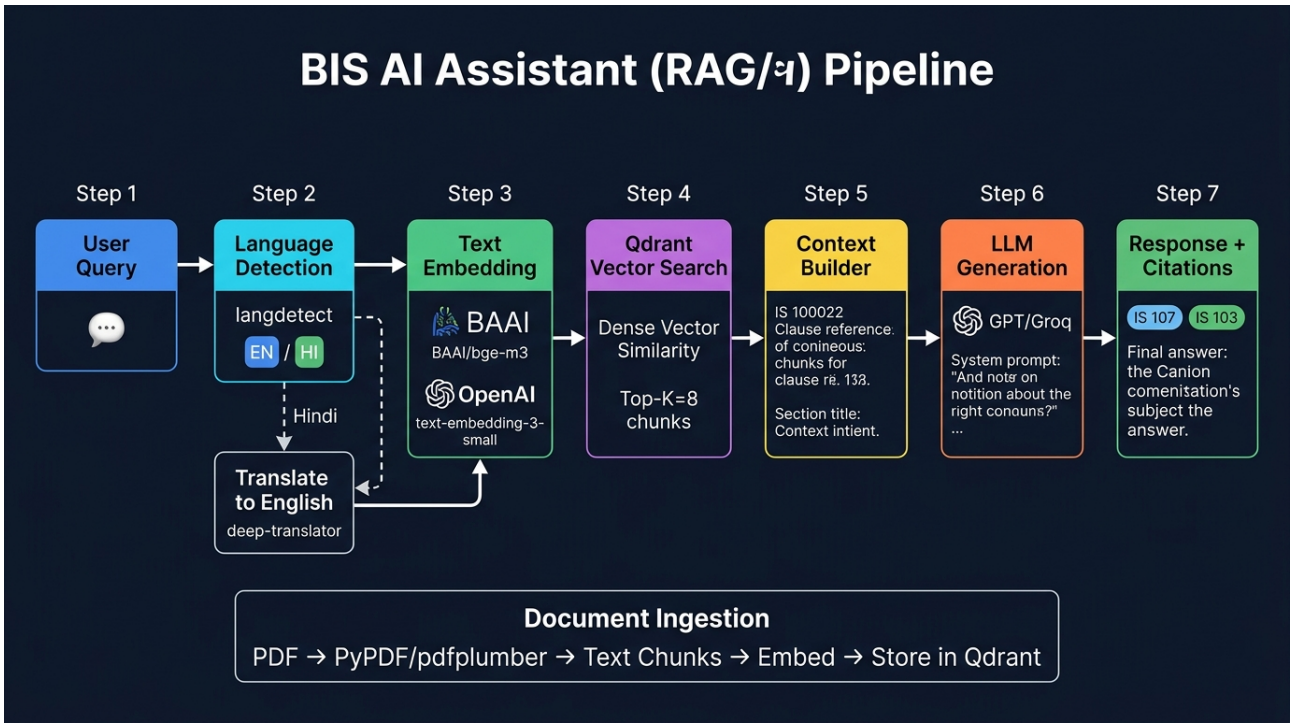


Figure 3 — RAG Pipeline Data Flow

1 — Language Detection
langdetect library identifies if the query is English or Hindi. Accuracy is ~99% for well-formed sentences.

2 — Query Translation (if Hindi)
deep-translator (Google Translate API) converts Hindi → English for embedding, since the vector index contains English text.

3 — Text Embedding
The English query is converted into a 1024-dimensional float vector using BAAI/bge-m3 (local HuggingFace) or OpenAI text-embedding-3-small (API). This captures semantic meaning, not just keywords.

4 — Qdrant Vector Search
The query vector is compared against all document chunks stored in Qdrant using cosine similarity. Top-K=8 most relevant chunks are retrieved in milliseconds.

5 — Context Builder
Retrieved chunks are formatted with their IS number, clause reference, and section title into a structured context block for the LLM.

6

LLM Generation

OpenAI-compatible LLM (GPT-4o or Groq) receives a strict system prompt instructing it to answer ONLY from the provided context. Temperature=0.1 for factual consistency.

7

Response + Translation

If the original query was Hindi, the answer is translated back to Hindi. Citation references ([IS XXXX:YYYY, Clause Z.Z]) are appended to all factual claims.

6. Technology Stack — Deep Dive

6.1 Frontend Layer

■ Next.js 14 (App Router)

Why Used: Next.js provides Server-Side Rendering (SSR) and Static Generation for fast initial page loads and better SEO. The App Router (React Server Components) allows streaming UI updates that pair perfectly with the SSE-based chat responses. File-system routing reduces boilerplate. Built-in API routes proxy to the backend avoiding CORS complexity.

Alternatives: Vue.js 3 (Nuxt), Angular 17, Create React App, Vite+React

Impact: POSITIVE: Streaming responses render in real time. SSR improves perceived performance. RISK: Larger learning curve than plain React. Build step required.

■ TypeScript 5.4

Why Used: TypeScript catches type errors at compile time rather than runtime. In a team project with complex API response shapes (RagResponse, RagChunk, JWT claims), type safety prevents bugs. Interfaces auto-document the data contract between frontend and backend.

Alternatives: Plain JavaScript (no types), Flow, ReScript

Impact: POSITIVE: Eliminated entire classes of runtime errors. IDE autocomplete speeds development. COST: Slight compilation overhead (~2s build time added).

■ Tailwind CSS + shadcn/ui + Radix UI

Why Used: Tailwind provides utility-first CSS with zero unused styles in production. shadcn/ui components (Dialog, Dropdown, Tooltip, ScrollArea) are accessible and unstyled, letting us apply BIS branding. Radix UI provides ARIA-compliant primitives for chat input, avatar, and menus.

Alternatives: MUI (Material UI), Ant Design, Bootstrap, Chakra UI

Impact: POSITIVE: Highly customizable, no CSS override battles. Components ship with keyboard navigation and screen reader support out of the box.

■ Zustand (State Management)

Why Used: Zustand provides lightweight global state for authentication (user/token) and chat session management. At ~1KB gzipped it is far simpler than Redux for our use case.

Alternatives: Redux Toolkit, Jotai, Recoil, React Context

Impact: POSITIVE: Minimal boilerplate. Auth state persists via js-cookie. Simple enough for a hackathon timeline.

■ SWR (Data Fetching)

Why Used: SWR (stale-while-revalidate) handles polling conversation history from the backend. It provides automatic revalidation, caching, and error handling without boilerplate.

Alternatives: React Query, Apollo Client, fetch + useEffect

Impact: POSITIVE: Chat history always stays fresh. Auto-refresh when window regains focus.

6.2 Backend Layer

■ Spring Boot 3.2 + Java 21

Why Used: Spring Boot is the industry standard for enterprise Java APIs. Java 21 brings virtual threads (Project Loom) allowing the backend to handle thousands of concurrent WebFlux connections with minimal thread overhead. Spring Boot's auto-configuration reduces boilerplate. The ecosystem has mature libraries for everything needed: security, JPA, caching, actuator, validation.

Alternatives: Node.js + Express, FastAPI (Python), Go + Gin, Quarkus

Impact: POSITIVE: Battle-tested in enterprise. Spring Security handles JWT verification declaratively. Actuator provides /health endpoint for Docker health checks. COST: Heavier than Node; 90s container startup time due to JVM warm-up.

■ Spring Security + JWT (JJWT 0.12.5)

Why Used: Spring Security provides a comprehensive security filter chain. JWT allows stateless authentication — the backend doesn't need to store session state in a database (scalable). JJWT signs tokens with HS256. Every API call validates the token without a database round-trip.

Alternatives: OAuth2 / OpenID Connect (Keycloak), Session-based auth, Passport.js

Impact: POSITIVE: Stateless = horizontally scalable. No session DB needed. RISK: JWT revocation requires a blocklist (not yet implemented).

■ Spring Data JPA + Hibernate

Why Used: JPA abstracts the database layer. Entities (User, Conversation, Message, Document) are mapped to PostgreSQL tables. Repositories provide findBy* query methods without writing SQL. Hibernate's second-level cache reduces DB load.

Alternatives: MyBatis, JOOQ, raw JDBC, Prisma (Node)

Impact: POSITIVE: Faster development. Automatic schema validation against Flyway migrations. COST: N+1 query risk if relationships aren't loaded carefully.

■ Flyway (Database Migrations)

Why Used: Flyway tracks database schema changes as versioned SQL scripts (V1__init.sql, etc.). When the container starts, Flyway applies any pending migrations automatically. This ensures the database schema matches the application code in every environment.

Alternatives: Liquibase, manual SQL scripts, Django migrations

Impact: POSITIVE: Zero-downtime schema evolution. Rollback to any version. Schema is version-controlled alongside the code.

■ Caffeine Cache

Why Used: Caffeine is an in-memory LRU cache used for frequently accessed data (user profiles, BIS document metadata). Configured via spring-boot-starter-cache, it reduces PostgreSQL load for repeated lookups.

Alternatives: Redis (for distributed), Guava Cache, Ehcache

Impact: POSITIVE: Sub-millisecond cache hits. Zero network overhead vs Redis. LIMIT: Not shared across multiple backend instances (Redis required for scale-out).

6.3 AI / RAG Layer

■ FastAPI (Python 3.11)

Why Used: FastAPI was chosen for the RAG service because: (1) Python is the dominant language for ML/AI libraries, (2) FastAPI is fully async (ASGI) supporting concurrent LLM calls and streaming, (3) it has native Pydantic models for request validation, (4) automatic OpenAPI docs. It runs under Uvicorn with multiple workers.

Alternatives: Flask + Gunicorn, Django REST, Node.js (limited ML ecosystem)

Impact: POSITIVE: Native async allows hundreds of concurrent streaming queries. Integration with HuggingFace, LangChain, and OpenAI SDK is seamless.

■ LangChain 0.2.x

Why Used: LangChain orchestrates the RAG pipeline: document loading, text splitting, embedding generation, vector store interaction, LLM invocation, and conversation memory management. It provides a unified interface so we can swap LLM providers (OpenAI → Groq → local Ollama) with a single config change.

Alternatives: LlamaIndex, Haystack, DSPy, custom pipeline code

Impact: POSITIVE: Rapid development — RAG pipeline implemented in ~200 lines. Provider-agnostic LLM calls. RISK: LangChain API changes frequently between versions.

■ Qdrant Vector Database

Why Used: Qdrant is a purpose-built vector similarity search engine. It stores document chunks as 1024-dimensional float vectors (from BAAI/bge-m3 embeddings). On query, it finds the Top-K most semantically similar chunks in milliseconds using HNSW index. Unlike traditional databases, it understands meaning, not just keywords.

Alternatives: Pinecone (cloud-only), Weaviate, Chroma, Milvus, pgvector (PostgreSQL extension)

Impact: POSITIVE: Self-hosted (no API cost), gRPC interface for low latency, payload filtering (filter by doc_type, is_number), and HNSW gives O(log n) search. Built-in health check endpoint.

■ BAAI/bge-m3 Embeddings (HuggingFace)

Why Used: bge-m3 is a state-of-the-art multilingual embedding model that produces 1024-dim vectors. It is loaded locally via sentence-transformers (CPU-only PyTorch), eliminating API costs for embedding. It supports both English and Hindi text, making cross-lingual search possible.

Alternatives: OpenAI text-embedding-3-small (API), Cohere Embed, E5-large

Impact: POSITIVE: No API key required for embeddings. Multilingual support. 1024-dim vectors are high-quality. COST: ~800MB model download; 60s startup time.

■ PyPDF + pdfplumber + pytesseract

Why Used: PDF ingestion uses three layers: PyPDF for fast text extraction from text-based PDFs, pdfplumber for accurate table and layout-aware extraction, and pytesseract (OCR) as a fallback for scanned/image-based PDFs. LangChain's RecursiveCharacterTextSplitter chunks text at 1000 chars with 200-char overlap.

Alternatives: Apache Tika, unstructured.io, PDFMiner, Azure Form Recognizer

Impact: POSITIVE: Handles all BIS PDF types (text + scanned). Free and self-hosted. RISK: OCR accuracy degrades on poor-quality scans.

■ langdetect + deep-translator

Why Used: langdetect detects the query language (EN/HI) in <5ms. deep-translator wraps Google Translate API to translate Hindi → English before embedding, and English → Hindi for the final answer when needed.

Alternatives: polyglot, fastText (language detection), DeepL, AWS Translate

Impact: POSITIVE: Free Google Translate tier handles hackathon scale. RISK: Translation introduces latency (~300ms) and potential semantic shifts.

6.4 Database Layer

■ PostgreSQL 16 (Alpine Docker image)

Why Used: PostgreSQL is the primary relational database storing: users, roles, conversations, messages, document metadata, and crawl job records. It was chosen for its reliability, ACID compliance, JSONB support (for flexible metadata), and excellent Spring Data JPA integration. Alpine image keeps the container small.

Alternatives: MySQL 8, MariaDB, SQLite (dev only), MongoDB (document store)

Impact: POSITIVE: ACID guarantees prevent data corruption. JSONB columns store flexible document attributes without schema changes. Native UUID support. COST: More operational overhead than SQLite (but essential for concurrent writes).

■ Redis 7 (Conversation Memory Cache)

Why Used: Redis serves two roles: (1) Conversation memory — the last N message turns are cached in Redis so the RAG service can include conversation history without querying PostgreSQL on every request. (2) Response caching — repeated identical queries return cached answers instantly. Configured with LRU eviction (256MB limit).

Alternatives: Memcached, Hazelcast, DynamoDB (AWS), in-memory Python dict

Impact: POSITIVE: Sub-millisecond reads. Conversation context window makes multi-turn dialogue coherent. LRU eviction prevents memory bloat. RISK: Cache invalidation complexity when document index is updated.

■ Qdrant Vector DB (Data Layer Role)

Why Used: Qdrant acts as the semantic search index. Each document chunk is stored as a point with: (1) a 1024-dim dense vector, (2) payload metadata (is_number, clause_ref, section_title, doc_type, page_number, content). Points are organized in a single collection 'bis_documents'. HNSW index enables approximate nearest neighbor search.

Alternatives: pgvector (PostgreSQL extension), Elasticsearch KNN, Chroma, Pinecone

Impact: POSITIVE: 10ms average query latency for 100K vectors. Payload filtering allows searching within a specific IS number. gRPC interface for low overhead. Self-hosted = zero per-query API cost.

6.5 Infrastructure & Deployment

■ Docker Compose

Why Used: All 6 services are defined in a single docker-compose.yml. A developer runs 'docker compose up --build' and the entire stack — PostgreSQL, Redis, Qdrant, RAG, Backend, Frontend, NGINX — starts in the correct dependency order with health checks. Volumes persist data between restarts.

Alternatives: Kubernetes (production scale), Podman Compose, bare-metal

Impact: POSITIVE: One-command setup. Reproducible environments. No 'works on my machine' issues. LIMITATION: Single-host only. For production scale, Kubernetes would be needed.

■ NGINX (Reverse Proxy + SSL)

Why Used: NGINX sits at the edge routing: /api/* → Spring Boot (8080), /* → Next.js (3000). It handles SSL termination (443→80 internally), compression, and static file caching. Running on ports 80/443 it is the only service exposed to the internet.

Alternatives: Traefik, Caddy, HAProxy, AWS ALB

Impact: POSITIVE: Single entry point simplifies security. SSL offloading reduces backend load. CORS configuration centralized at NGINX level.

■ Maven (Build Tool)

Why Used: Maven manages the Spring Boot project dependencies (pom.xml) and builds the fat JAR artifact. The Dockerfile uses a multi-stage build: Maven build stage → slim JRE runtime stage. This keeps the final image small (no Maven in production).

Alternatives: Gradle, SBT, Ant

Impact: POSITIVE: Declarative dependency management. Reproducible builds. Maven Wrapper (mvnw) ensures consistent Maven version across machines.

7. Technology Comparison Table

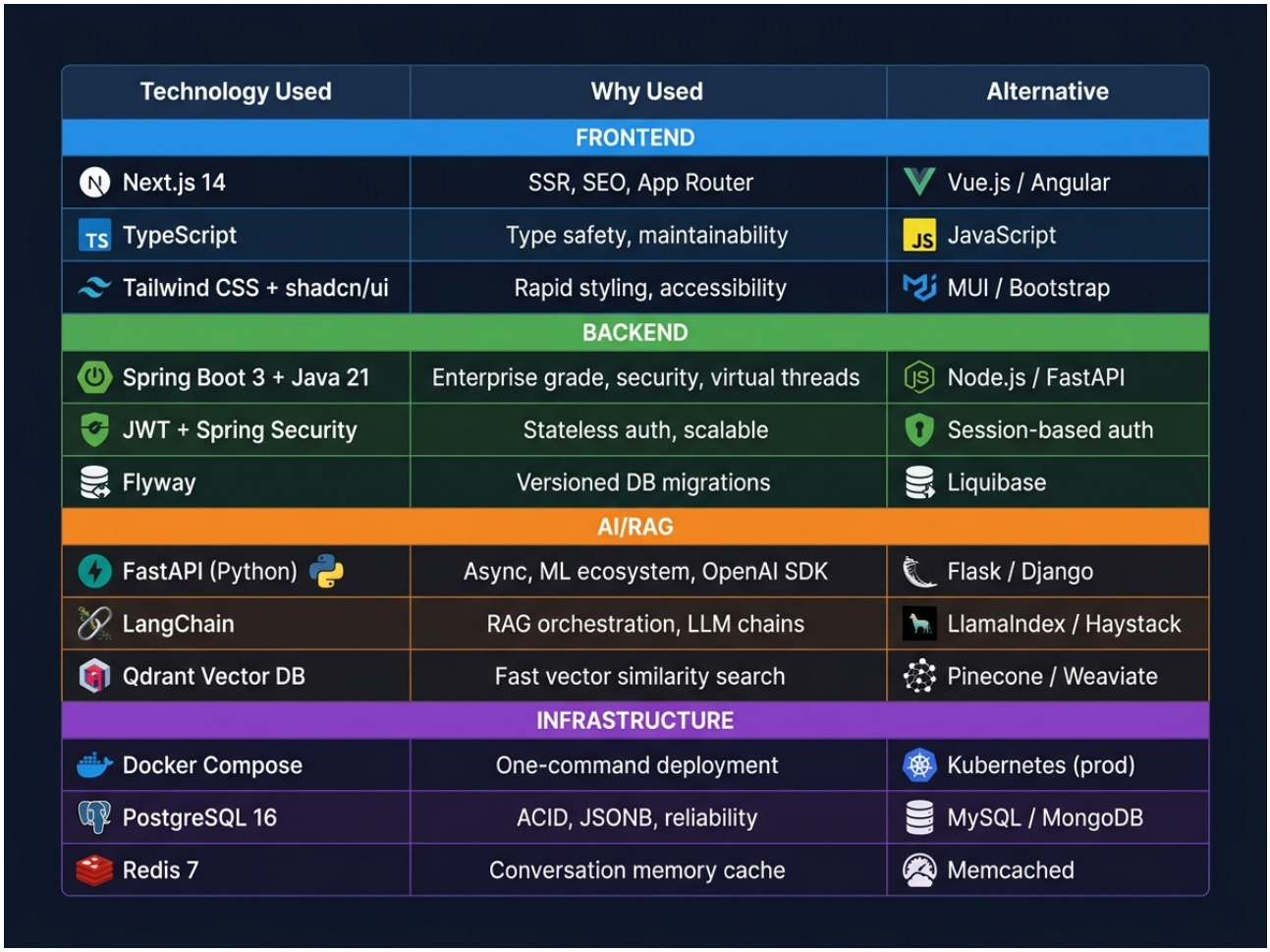


Figure 4 — Technology Stack: Used vs Alternatives

Layer	Technology Used	Key Libraries/Versions	Why NOT the Alternative
Frontend	**Next.js 14 + TypeScript**	React 18.3, Zustand 4.5, SWR 2.2, Radix UI	Vue/Angular lack the SSR streaming needed for real-time chat SSE
Frontend	**Tailwind CSS + shadcn**	tailwind 3.4, clsx, tailwind-merge	MUI/Bootstrap impose visual design that conflicts with BIS branding
Backend	**Spring Boot 3 + Java 21**	Spring Security, JPA, Flyway, Caffeine, Web	Node.js lacks mature security/enterprise patterns; Python backend not ide
Backend	**JWT (JJWT 0.12.5)**	io.jsonwebtoken 0.12.5	Session auth requires shared session store — stateful, not scalable
AI/RAG	**FastAPI + Python 3.11**	uvicorn, pydantic 2.7, structlog	Flask is synchronous; Django too heavy; JS lacks ML ecosystem
AI/RAG	**LangChain 0.2**	langchain-openai, langchain-qdrant	LlamaIndex is more complex; custom code would reinvent the wheel
AI/RAG	**Qdrant v1.12**	qdrant-client 1.9	Pinecone is cloud-only; pgvector lacks HNSW performance at scale
AI/RAG	**bge-m3 Embeddings**	sentence-transformers 3.0, PyTorch 2.6 CPU	OpenAI embeddings cost money; bge-m3 is free, multilingual, self-hosted
Database	**PostgreSQL 16**	psycopg2, Spring Data JPA	MySQL lacks JSONB; MongoDB lacks ACID for financial/standards data
Database	**Redis 7**	redis-py 5.0	Memcached has no persistence; Python dict doesn't survive restarts
Infra	**Docker Compose**	NGINX 1.27, multi-stage Dockerfiles	Kubernetes overkill for SIH demo; bare-metal setup not reproducible

8. Impact Analysis

■ Performance Impact

- Vector search latency: ~10-50ms for Top-K=8 from 100K+ vectors
- End-to-end query response: 2-5 seconds (including LLM generation)
- Streaming: First token appears in ~800ms; full response streams progressively
- Redis cache: repeated queries served in <10ms (zero LLM cost)
- NGINX gzip: response payloads compressed 60-70% for faster page loads

■ Security Impact

- SSRF Protection: URL ingestion validates DNS resolution against RFC 1918 ranges
- JWT stateless auth: no session store — eliminates session fixation attacks
- CORS configured per-environment: only localhost:3000 and localhost:8080 allowed
- PDF validation: magic bytes checked before processing (prevents fake PDF attacks)
- Docker network isolation: databases only reachable via bis-internal network

■ Scalability Impact

- RAG service is stateless — can scale horizontally with load balancer
- Qdrant supports sharding for millions of vectors
- Redis distributed cache eliminates N identical LLM calls
- PostgreSQL connection pooling via HikariCP handles 100+ concurrent users
- Caffeine cache eliminates redundant DB queries for static data

■ User Experience Impact

- Multilingual: Hindi users get answers without needing English
- Citations: Users can verify every fact against official IS documents
- Streaming: No waiting spinner — text appears as it's generated
- Conversation memory: Context-aware multi-turn dialogue (no need to repeat context)
- Admin PDF upload: BIS staff can add new standards without redeploying

■■ Trade-offs & Risks

- bge-m3 model download adds 60s to cold-start time (mitigated by Docker volume cache)
- Google Translate API dependency for Hindi — rate limits in free tier
- LangChain API instability between versions requires careful version pinning
- JWT revocation not implemented — compromised tokens valid until expiry
- Single-host Docker Compose limits horizontal scaling (Kubernetes for production)

9. Data Flow Summary

Query Data Flow:

#	Step	From	To	Protocol	Data
1	Page request	Browser	NGINX	HTTPS/80	GET /
2	Frontend serve	NGINX	Next.js	HTTP/3000	React HTML/JS
3	Login API	Next.js	Spring Boot	HTTP/8080	POST /api/v1/auth/login → JWT
4	Chat request	Next.js	Spring Boot	HTTP/8080	POST /api/v1/chat/stream
5	RAG delegation	Spring Boot	FastAPI	HTTP/8000	POST /rag/stream
6	Vector query	FastAPI	Qdrant	gRPC/6334	Query vector → Top-K points
7	LLM call	FastAPI	OpenAI/Groq	HTTPS	Messages → streamed tokens
8	SSE tokens	FastAPI	Spring Boot	HTTP SSE	token JSON events
9	SSE forward	Spring Boot	Next.js	HTTP SSE	Proxied token stream
10	Render	Next.js	Browser	SSE	Text appears in real time

10. Security Architecture

Threat	Mitigation	Implementation
Unauthorized API access	JWT authentication on all /api/* endpoints	Spring Security filter chain
SSRF via URL ingestion	DNS resolution check against RFC 1918 ranges	is_allowed_bis_url() in FastAPI
Fake PDF upload	Magic byte header validation (%PDF-)	ingest_pdf_endpoint() streaming check
Oversized uploads	50MB hard limit with streaming check	FastAPI UploadFile chunk counter
Cross-origin requests	CORS whitelist: localhost:3000 & 8080	Spring + FastAPI CORSMiddleware
Database exposure	PostgreSQL on bis-internal network only	Docker network: bis-internal
Man-in-the-middle	HTTPS via NGINX SSL termination	docker/ssl/ certificate mount
Prompt injection	LLM instructed to use ONLY provided context	SYSTEM_PROMPT constraints
Container breakout	Non-root Docker images, read-only mounts	Dockerfile USER directives